

Laboratório II

Bento Rabello Botega (186461)

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada (`ARRIVAL_DELAY > 10`)?

Uma vez que o percentual é dado por $\frac{n_{\text{atrasos}}}{n}$, informação sobre a quantidade total de vôos, `n`, e quantidade de vôos atrasados, `n_delay` é suficiente.

2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):
 1. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
 2. Remova observações que tenham valores faltantes em campos de interesse;
 3. Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
 4. Para cada grupo em b., determine as estatísticas apontadas no item 1. e os retorne como um objeto da classe `tibble`;
5. A função deve receber apenas dois argumentos:
 1. `input`: o conjunto de dados (referente ao lote em questão);
 2. `pos`: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. **É importante** observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
library("dplyr")
```

```
Attaching package: 'dplyr'
```

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
getStats <- function(input, pos) {  
  input |>  
  filter(AIRLINE %in% c('AA', 'DL', 'UA', 'US')) |>  
  filter(!is.na(ARRIVAL_DELAY), !is.na(MONTH), !is.na(DAY)) |>  
  group_by(DAY, MONTH, AIRLINE) |>  
  summarise(  
    n_delay = sum(ARRIVAL_DELAY > 10),  
    n = n(),  
    .groups = "drop"  
  )  
}
```

3. Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.

1. Configure o tamanho do lote (chunk) para 100 mil registros;
2. Configure a função de *callback* para instanciar DataFrames aplicando a função `getStats` criada acima;
3. Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```
library("readr")  
  
chunks <- read_csv_chunked(  
  file = "../datasets/flights.csv",  
  chunk_size = 100000,  
  callback = DataFrameCallback$new(getStats),  
  col_types = cols_only(  
    DAY = col_integer(),  
    MONTH = col_integer(),  
    AIRLINE = col_character(),  
    ARRIVAL_DELAY = col_double()
```

```
)  
)
```

4. Crie uma função chamada `computeStats` que:

1. Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
2. Retorne as informações em um `tibble` contendo apenas as seguintes colunas:
 1. `Cia`: sigla da companhia aérea;
 2. `Data`: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`);
 3. `Perc`: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo $[0,1]$.

```
computeStats <- function(stats) {  
  stats |>  
    group_by(DAY, MONTH, AIRLINE) |>  
    summarise(  
      n_delay = sum(n_delay),  
      n = sum(n),  
      .groups = "drop"  
    ) |>  
    mutate(  
      Cia = AIRLINE,  
      Data = as.Date(paste(2015, MONTH, DAY, sep='-')),  
      Perc = n_delay / n  
    ) |>  
    select(Cia, Data, Perc)  
}
```

```
stats <- computeStats(chunks)  
stats
```

```
# A tibble: 1,276 x 3  
  Cia    Data      Perc  
  <chr> <date>    <dbl>  
1 AA    2015-01-01 0.373  
2 DL    2015-01-01 0.0847  
3 UA    2015-01-01 0.289  
4 US    2015-01-01 0.196  
5 AA    2015-02-01 0.201
```

```
6 DL      2015-02-01 0.203
7 UA      2015-02-01 0.256
8 US      2015-02-01 0.327
9 AA      2015-03-01 0.541
10 DL     2015-03-01 0.372
# i 1,266 more rows
```

```
# install.packages("remotes")
# remotes::install_github("jayjacobs/ggcal")
```

```
library("ggcal")
library("ggplot2")

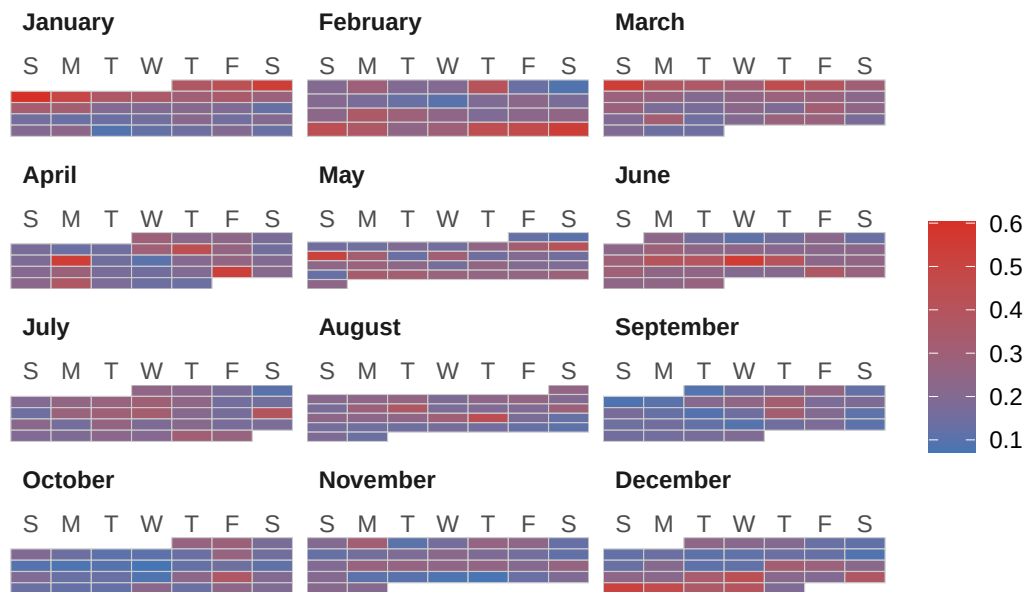
pal <- scale_fill_gradient(low = "#4575b4", high = "#d73027")

baseCalendario <- function(stats, cia) {
  local_stats <- stats |>
    filter(Cia == cia)
  return(ggcal(local_stats$Data, local_stats$Perc))
}

cAA <- baseCalendario(stats, "AA")
cDL <- baseCalendario(stats, "DL")
cUA <- baseCalendario(stats, "UA")
cUS <- baseCalendario(stats, "US")

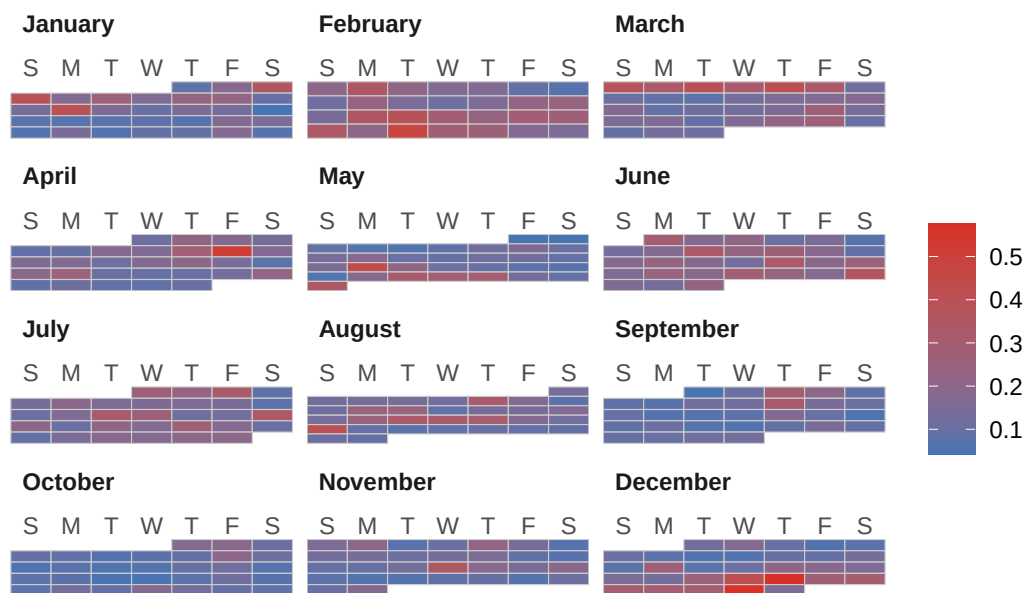
cAA + pal + ggtitle("Percentual de atrasos diários AA")
```

Percentual de atrasos diários AA



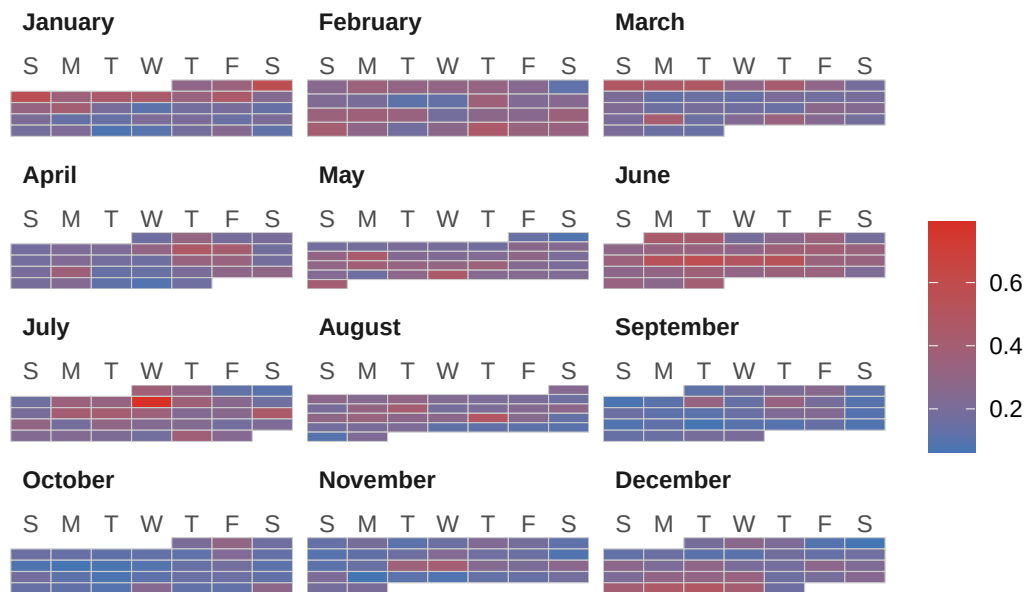
```
cDL + pal + ggtitle("Percentual de atrasos diários DL")
```

Percentual de atrasos diários DL



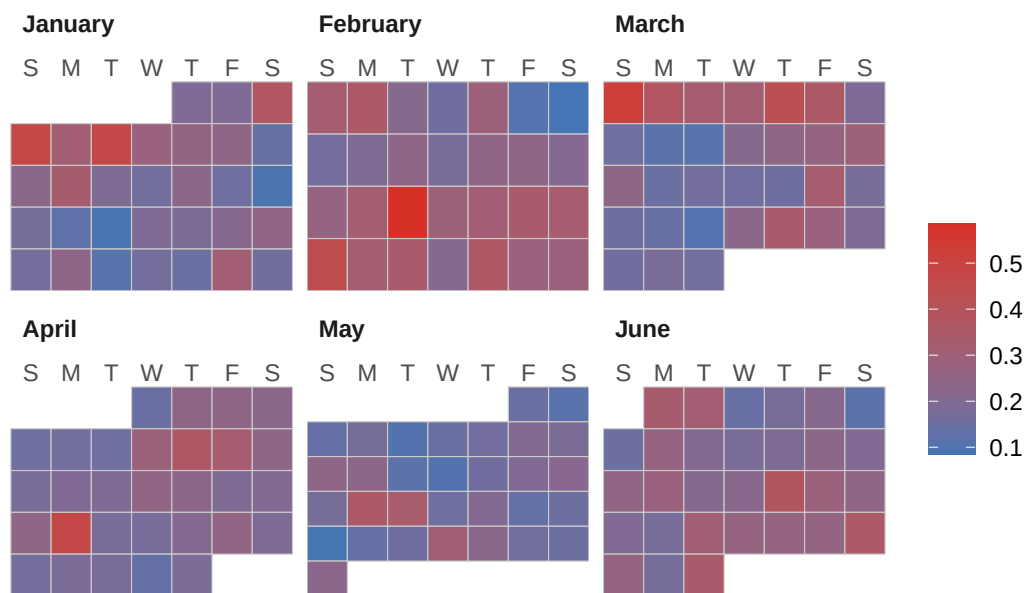
```
cUA + pal + ggtitle("Percentual de atrasos diários UA")
```

Percentual de atrasos diários UA



```
cUS + pal + ggtitle("Percentual de atrasos diários US")
```

Percentual de atrasos diários US



Momento de compilação

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-21 20:33:34 -03"
```